COMP 018: Database Administration - Activity 6 Progress Report
Database Backup Planning, Verification, Restoration, and Disaster Recovery Simulation

Project Identification & Metadata

Course	: COMP 018 - Database Administration
Evaluating Group	: Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	: https://github.com/Nycri/brewpos
Target Project Monitored	: Park-It: Parking Management System (Park-It Database Management System)
Target Project Group	: Group 2: Miller Devera, Jhon Francis, et al.
Target Project Repository	: https://github.com/millerdevera6-glitch/park-it-database
Primary Database	: grp_parkitdb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	: grp_parkitdb_restore_test

ACTIVITY 6: Database Backup, Restoration, and Recovery

3.1 Backup Planning & Architecture
The disaster recovery plan for the Park-It database encompasses five relational tables within the public schema, capturing parking zone definitions, vehicle registries, staff accounts, live check-ins, and financial payment receipts. The plan covers sequence counters, primary keys, foreign keys, constraints, and privilege grants to ensure operational continuity.

The backup strategy utilizes PostgreSQL custom compressed archive format via a dedicated backup operator role. Archive dumps are stored in a protected directory outside web roots and repository trees. Recovery objectives define a Recovery Point Objective of 30 minutes and a Recovery Time Objective of 1 hour. Retention guidelines specify maintaining seven daily backups alongside four weekly snapshots.

Because database login accounts exist at the PostgreSQL cluster level and are not stored within single-database dumps, disaster recovery plans require executing access-control role scripts on new server instances prior to restoring database contents.

3.2 Backup Creation, Checksum & Archive Inspection
Logical backup creation was executed using the dedicated backup account, producing a compressed custom format archive sized at 19,750 bytes. Cryptographic verification computed a SHA-256 hash value of 7F836A870824269350C868AB8D95EE89A17741D810CDFE8284483D87FB2F4A35, establishing a baseline for integrity verification.

Inspection of the archive table of contents verified 60 catalog entries, confirming the inclusion of all five database tables, sequence objects, primary keys, check constraints, foreign keys, and default privilege grants. To ensure recoverability beyond file inspection checks, a complete restoration drill was conducted in an isolated test database.

3.3 Restoration into Separate Test Database
An isolated test database named grp_parkitdb_restore_test was created on the local server instance. The custom archive dump file was restored into this database using administrative utility tools. The restoration process completed cleanly with zero error codes.

3.4 Post-Restore Verification Suite
Post-restoration verification confirmed that all five relational tables were recreated with proper schema structure under administrative ownership. Record count validation across tables demonstrated complete parity with the source database, matching five parking zones, five vehicles, four staff accounts, five transactions, and three payment receipts. Database checks confirmed that all 36 constraints were fully restored.

Access control verification on the restored test database confirmed that privilege definitions transferred accurately. Queries executed under the application runtime account successfully returned parking zone records, while query attempts by the read-only auditing account on staff credential tables were blocked by the database security layer.

3.5 Controlled Recovery Simulation
A controlled data recovery simulation was performed to test incident response procedures. A disposable parking zone record was inserted into the restored test database to establish a baseline state. A data loss event was then simulated by removing the record, and queries confirmed its deletion.

Targeted record restoration was executed to reinsert the missing data row without affecting existing database entries. Post-recovery verification confirmed that the record was fully restored with its original attributes intact. Following verification, the test record was removed to clean up the test environment.

3.6 Limitations, Evaluation & Production Recommendations
Restoration procedures on fresh server installations require pre-executing role definition scripts to establish user accounts before restoring database ACLs. For production operations, daily automated backup generation should be scheduled using system task schedulers to archive database dumps with date-stamped filenames and retain snapshots according to policy. Enabling continuous write-ahead log archiving will support point-in-time recovery capabilities.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 6)

Column Header	Value
Project	Group 2 - Park-It: Parking Management System
Database and backup file	grp_parkitdb, grp_parkitdb_activity6.dump, Size: 19,750 bytes, SHA256: 7F836A870824269350C868AB8D95EE89A17741D810CDFE8284483D87FB2F4A35
Backup method	pg_dump custom format (-Fc), compressed binary dump (2026-11-05 03:50:58)
Restore test results	Success (Restored into grp_parkitdb_restore_test; all 5 tables, constraints, and data verified)
Access control after restore	(Activity 4 roles and permissions validated; parkit_report_user denied reading staff_user)
Recovery simulation result	Recovered (Simulated deleted test parking zone successfully restored from baseline state)
Proposed RPO and RTO	RPO: 30 minutes, RTO: 1 hour
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-6/Evidence-act6
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity6_Progress_Report_Group2.pdf
Last Updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.